Spécification fonctionnelle et technique – Plateforme de défis informatiques

Nom de travail : **CodeQuest** · Version 1.0 (brouillon) · 23 octobre 2026

1. Contexte et objectifs

CodeQuest est une plateforme web d'apprentissage et de compétition où des développeurs, seuls ou en équipe, relèvent des défis informatiques : algorithmique (tris de listes), débogage, cryptographie, stéganographie et QR codes. Le tout est habillé par un univers narratif (lore), un système de progression, des événements limités dans le temps, des boss et une économie virtuelle.

Objectifs

- Apprendre par la pratique : chaque notion est d'abord enseignée (didacticiel), puis éprouvée par un défi.
- Motiver : score, classements (joueurs et équipes), récompenses, points de vie, boss, battle royale.
- Socialiser : équipes, édition collaborative, chat textuel et vocal.
- Livrer un projet professionnel : conteneurisé, versionné, testé, sécurisé, accessible sur tous les appareils.

Hors périmètre (v1) : paiements en argent réel, application mobile native, support de plus de 3 langages de programmation, tournois en présentiel.

2. Priorisation (MoSCoW)

Priorité	Fonctionnalités
Must	Authentification complète, rôles, défis de tri avec scoring, classements, base de données, API, interface web, conteneurisation, versionnage, tests, gestion des secrets, thématiques, prérequis
Should	Équipes, events (1 essai), défis de débogage et de décryptage, didacticiels, monnaie virtuelle, lore, DA soignée, son
Could	Boss et points de vie, stéganographie, QR code caché, chat textuel, battle royale

Priorité	Fonctionnalités
Won't (v1)	Chat vocal avancé (salons multiples, enregistrement), app native

Le chat vocal et le battle royale sont classés en phases tardives (voir section 15) car ils dépendent de la stabilité du socle temps réel.

3. Acteurs et rôles

Rôle	Droits
Visiteur	Voir la page d'accueil, le lore public, le classement public ; s'inscrire ou se connecter
Joueur (utilisateur)	Suivre les cours, résoudre les défis, rejoindre ou créer une équipe, participer aux events, utiliser le chat, gérer sa monnaie, modifier ou supprimer son compte
Capitaine d'équipe	Droits joueur + gérer les membres, choisir la soumission officielle de l'équipe, inscrire l'équipe aux events
Modérateur	Droits joueur + modérer le chat (mute, suppression, signalements)
Administrateur	Tout : CRUD défis, cours, events, boss, lore, boutique ; gestion des utilisateurs et des rôles ; journal d'audit ; paramètres globaux

La gestion des rôles repose sur un modèle RBAC : un utilisateur peut cumuler plusieurs rôles, chaque rôle porte un ensemble de permissions (par ex. `challenge:write`, `user:ban`, `chat:moderate`). Le rôle « capitaine » est contextuel (propre à une équipe), les autres sont globaux.

4. Spécification fonctionnelle

4.1 Compte et authentification

- **Inscription** : e-mail, pseudo unique (3–20 caractères), mot de passe (12 caractères minimum, vérification contre une liste de mots de passe compromis). Un e-mail de confirmation est envoyé ; le compte reste limité tant qu'il n'est pas confirmé.

- **Connexion** : e-mail ou pseudo + mot de passe. Message d'erreur générique (ne révèle pas quel champ est faux). Blocage temporaire progressif après 5 échecs.
- **Mot de passe oublié** : demande par e-mail, envoi d'un lien contenant un jeton à usage unique valable 1 heure ; la réponse de l'API est identique que l'adresse existe ou non. Toutes les sessions actives sont révoquées après changement du mot de passe.
- **Déconnexion** : révoque la session courante ; option « se déconnecter partout ».
- **Effacer son compte** : confirmation par mot de passe, puis suppression logique immédiate (compte inaccessible) et suppression définitive après 30 jours. Les données personnelles sont effacées ; les scores et messages sont anonymisés (« Joueur supprimé ») pour préserver l'intégrité des classements. Si le joueur est capitaine, la capitainerie est transférée automatiquement.
- **Profil** : avatar, pseudo, langage préféré, préférences son et thème, historique, badges. Export des données personnelles (RGPD).

4.2 Types de défis

Chaque défi appartient à une **thématique**, a une **difficulté** (1 à 5), un **type**, un **énoncé**, des **tests** (visibles et cachés), un **barème** et éventuellement des **prérequis**.

a) Tri de listes (algorithmique)

- Le joueur écrit une fonction (Python, JavaScript ; Java ou C++ ensuite) qui trie ou manipule une liste : tri à bulles, par insertion, fusion, rapide, par tas, tri avec comparateur personnalisé, tris stables, tri de structures, tri de grands volumes.
- Le code est exécuté dans un bac à sable sur des jeux de tests (petits, limites, très grands).
- Le score dépend de la **correction** (tous les tests doivent passer) puis du **temps d'exécution** (voir 4.3).
- Chaque défi de tri est précédé d'un cours (4.7) expliquant l'algorithme et sa complexité.

b) Débogage

- Un code contenant un ou plusieurs bugs (logique, erreur d'indice, cas limite, fuite d'état) est proposé dans l'éditeur. Le joueur le corrige jusqu'à ce que la suite de tests cachée passe.
- Variantes : trouver un bug unique, corriger N bugs, corriger sans dépasser un nombre de lignes modifiées (bonus).
- Score : correction + nombre d'essais + temps de résolution.

c) Décryptage de texte

- Un texte chiffré est fourni (César, Vigenère, XOR avec clé courte, substitution mono-alphabétique, Base64 imbriqué, chiffrements maison décrits dans le lore). Le joueur soumet le texte clair, soit à la main, soit via un script d'aide.
- Le texte et la clé sont **générés par instance** à partir d'une graine (seed) propre à chaque joueur ou équipe, pour empêcher le partage de solutions.

d) Texte caché dans une image (stéganographie)

- Le joueur reçoit une image contenant un message (LSB, canaux de couleur, métadonnées, texte quasi invisible). Il doit retrouver le message et le soumettre. Des indices progressifs (payants en monnaie virtuelle) sont disponibles.
- Variante inverse (mode « atelier ») : le joueur cache lui-même un message dans une image et la plateforme vérifie qu'un autre module (extracteur de référence) le retrouve.

e) QR code caché

- Un QR code est dissimulé (superposé avec faible contraste, fragmenté, caché dans les bits de poids faible, partiellement dégradé). Le joueur le retrouve, le décode et soumet le contenu.
- Variante « atelier » : le joueur génère un QR code valide qui contient une valeur imposée.

4.3 Moteur d'évaluation et scoring

Règle générale : un défi n'est validé que si **100 %** des tests passent. Le score se calcule ensuite ainsi :

```
Score = Points_base + Points_perf
```

- `Points_base` : fixé par le défi selon sa difficulté (par ex. 100 × difficulté).
- `Points_perf = Perf_max × clamp(1 − (t − t_opt) / (t_max − t_opt), 0, 1)`, où `t` est le temps d'exécution mesuré, `t_opt` le temps d'une solution de référence optimale et `t_max` le temps au-delà duquel aucun bonus n'est accordé.

Mesure du temps (point sensible) :

- Le temps est mesuré côté serveur, dans le bac à sable, en **temps CPU** (et non en temps mural) pour limiter l'effet de la charge de la machine.
- Chaque soumission est exécutée 5 fois ; on retient la **médiane**.
- Les ressources (CPU, mémoire) sont figées par défi. Les tests de référence (`t_opt`, `t_max`) sont recalibrés sur la même infrastructure.
- Une plage de tolérance (±5 %) évite qu'un score change pour une différence non significative.

Autres types de défis : le score dépend du temps de résolution (depuis l'ouverture du défi), du nombre d'indices utilisés et du nombre d'essais.

Essais :

- **Défis standard** : essais illimités, seul le **meilleur score** est conservé.
- **Events** : **1 seul essai** par joueur (ou par équipe). La soumission est définitive.
- **Boss** : nombre d'essais limité par les points de vie (4.5).

Classements

- Classement **joueurs** : global, par thématique, par saison, par event.
- Classement **équipes** : somme (ou moyenne pondérée) des meilleurs scores d'équipe.
- Filtres : période (semaine, mois, saison, tout temps), amis, équipe.
- Départage : à score égal, la soumission la plus ancienne gagne.
- Les classements sont calculés en temps quasi réel (Redis) et reconstruits périodiquement depuis la base pour garantir la cohérence.

4.4 Events

- Un event est un défi (ou une série de défis) **disponible uniquement sur une plage de dates** (début, fin, fuseau explicite).
- Hors plage, l'event est invisible ou verrouillé (aperçu « à venir »).
- **Un seul essai** : le joueur confirme explicitement avant de démarrer ; le chronomètre démarre à l'ouverture. Un abandon ou une fermeture du navigateur consomme l'essai.
- Un event dispose de son propre classement, de ses récompenses (monnaie, badge, titre) et d'un statut (brouillon, planifié, ouvert, terminé, archivé).
- Les résultats sont publiés à la fermeture ; les corrigés sont débloqués après la fin.

4.5 Boss et points de vie

Boss : un gros défi composé de plusieurs **phases**, chacune étant un sous-défi (par ex. un tri, un décryptage, un débogage). Le boss est lié à un chapitre du lore, a une barre de vie propre et des récompenses importantes.

Points de vie (PV) :

- Chaque joueur dispose de **100 PV max**.
- Les PV ne sont consommés que sur les **boss** (et certains events marqués « hardcore ») : un essai raté coûte des PV (par ex. 10 à 25 selon le boss).
- Les défis standard ne coûtent aucun PV (essais infinis).
- À 0 PV, le joueur est **KO** : accès au boss bloqué pendant une durée de récupération (par ex. 30 min).

- Régénération passive (par ex. 1 PV toutes les 6 minutes) ; régénération instantanée via un objet « potion » acheté avec la monnaie virtuelle.
- Les **PV du boss** diminuent à chaque phase réussie, proportionnellement au score obtenu. Le boss est vaincu quand ses PV tombent à 0.
- En équipe, les PV sont **partagés** (pool d'équipe) : les dégâts subis et infligés sont cumulés.

4.6 Équipes

- Création d'une équipe (nom unique, tag, description, logo), invitation par lien ou par pseudo, demande d'adhésion.
- Taille : 2 à 5 membres (configurable par défi ou event).
- Un joueur appartient à **une seule équipe active** à la fois ; il peut jouer en solo en parallèle.
- **Plusieurs développeurs sur le même défi** : édition collaborative du code en temps réel (curseurs visibles, éditeur partagé), console de résultats commune.
- La **soumission officielle** est déclenchée par le capitaine ; chaque membre peut soumettre des essais, mais seul le résultat retenu compte pour l'équipe.
- Répartition : le score d'équipe est celui de la soumission retenue ; un **score individuel de contribution** (nombre de lignes conservées, essais réussis, assistance) alimente le classement joueurs avec un coefficient réduit.
- Quitter ou être exclu ne retire pas les scores déjà acquis par l'équipe ; il retire le droit de contribuer aux défis en cours.

4.7 Progression, thématiques, prérequis et didacticiels

- **Thématiques** : Tris et listes, Débogage, Cryptographie, Stéganographie, QR codes, Boss. Un défi a une thématique principale et des tags secondaires. Filtrage et tri par thème, difficulté, statut (verrouillé, disponible, réussi), type (solo, équipe, event).
- **Prérequis** : un défi peut exiger la réussite d'autres défis, un niveau minimum, un cours terminé ou un chapitre du lore atteint. Représentation en **graphe orienté acyclique** (arbre de compétences). Un défi verrouillé affiche ce qui manque.
- **Didacticiels** : un cours par notion (leçon courte + exemples interactifs + mini-quiz). Il se termine par un défi d'application. Un cours est obligatoire avant le premier défi de sa notion mais peut être passé par un joueur confirmé (test de positionnement).
- **Niveaux et XP** : chaque défi réussi rapporte de l'expérience ; le niveau conditionne l'accès à certaines zones du lore.

4.8 Lore, direction artistique et son

Lore (proposition) : dans un futur proche, *ORACLE*, l'IA qui gère le Réseau mondial, a été corrompue. Les joueurs sont des « Runners » qui reprennent le contrôle de secteurs du réseau. Chaque thématique correspond à une région (le Dédale des Tris, la Crypte des Chiffres, l'Atelier des Fantômes pour la stéganographie…), chaque boss est un « Gardien » corrompu. Les chapitres sont débloqués par la progression et l'état de leur lecture est conservé.

Direction artistique :

- Esthétique rétro-futuriste « terminal néon » : fond sombre, accents cyan et magenta, typographie monospace pour le code, titres expressifs.
- Design system documenté (couleurs, typographie, espacements, composants) et mode clair en option.
- Animations sobres (barre de PV, apparition des boss, validation d'un défi), désactivables (`prefers-reduced-motion`).
- Accessibilité visée : WCAG 2.1 AA (contrastes, navigation clavier, lecteurs d'écran).

Son :

- Musique d'ambiance par région, effets sonores (succès, échec, coup porté au boss, KO, notification).
- Réglages : volume maître, musique, effets, chat vocal ; coupure globale. Le son ne démarre qu'après une interaction (contraintes d'autoplay des navigateurs).
- Chaque information importante véhiculée par le son doit aussi l'être visuellement.

4.9 Chat textuel et vocal

- **Canaux textuels** : global, par équipe, par event, messages privés (optionnel).
- Historique, mentions, liens sécurisés, limitation de débit (anti-spam), filtre de contenu, signalement.
- **Modération** : mute, suppression de message, bannissement temporaire par un modérateur ou un admin ; journalisation.
- **Chat vocal** : salons vocaux d'équipe et d'event via WebRTC ; push-to-talk ou détection de voix ; coupure du micro, volume par participant. Aucun enregistrement.

4.10 Monnaie virtuelle et récompenses

- Monnaie : les **Bits** (virtuelle, non convertible en argent réel).
- **Gains** : défi réussi (une fois par défi, plus un bonus si le record personnel est battu), event, boss, succès, série de connexions.

- **Dépenses** : indices, potions de PV, cosmétiques (avatars, titres, thèmes d'éditeur), accès anticipé à certains contenus non compétitifs.
- Les récompenses ne doivent pas rendre un joueur plus fort en compétition : les objets achetables n'augmentent pas les scores. Les indices sont désactivés en events.
- Tout mouvement est enregistré dans un **registre de transactions immuable** (idempotent) ; le solde est dérivé de ce registre.
- **Badges et succès** : « Premier tri », « 10 défis sans erreur », « Boss vaincu sans dégât », etc.

4.11 Battle royale

- Event spécial par **rounds**. À chaque round, tous les participants encore en vie relèvent un défi (même énoncé, 1 essai, durée limitée).
- À la fin du round, le participant (ou l'équipe) au **score le plus bas est éliminé** ; en cas d'égalité, on élimine celui qui a soumis le plus tard. Les éliminés passent en mode spectateur (chat spectateurs).
- Les rounds se poursuivent jusqu'à ce qu'il reste un vainqueur. Le nombre d'éliminations par round dépend du nombre de participants (par ex. 1 par round, ou 10 % si plus de 50 joueurs).
- Les rounds sont synchronisés par le serveur (compte à rebours commun, horloge serveur) ; une déconnexion ne met pas en pause le round.
- Minimum de participants fixé (par ex. 8) ; sinon l'event est reporté.

4.12 Administration

- Tableau de bord : utilisateurs actifs, défis les plus joués, taux de réussite, files d'exécution.
- CRUD des défis (avec versions), cours, thématiques, prérequis, events, boss, lore, boutique.
- Gestion des utilisateurs et des rôles (attribuer, retirer, suspendre, supprimer), gestion des signalements.
- Validation d'un défi avant publication : exécution automatique de la solution de référence, vérification des tests et calibrage de `t_opt` / `t_max`.
- Journal d'audit consultable (qui a fait quoi, quand).

5. Architecture technique

5.1 Vue d'ensemble

```
[Navigateur / PWA]  ──HTTPS──►  [Reverse proxy (Nginx / Traefik)]
                                    ├─► Front statique (React)
                                    ├─► API REST + WebSocket (backend)
                                    └─► Serveur média WebRTC (LiveKit)
[Backend] ──► PostgreSQL (données)
          ──► Redis (sessions, cache, classements, files, pub/sub)
          ──► Stockage objet S3/MinIO (images, assets)
          ──► File de jobs ──► [Runners sandbox isolés]
          ──► Service e-mail (SMTP)
```

5.2 Choix technologiques proposés

Couche	Choix	Justification
Front	React + TypeScript, Vite, Tailwind, Monaco Editor	Écosystème riche, éditeur de code de qualité
Temps réel	WebSocket (Socket.IO) + Yjs (CRDT)	Chat, édition collaborative, battle royale
Vocal	LiveKit (WebRTC)	SFU auto-hébergeable, évite de coder la signalisation
Back	Node.js + NestJS (TypeScript)	Même langage que le front, architecture modulaire, DI, OpenAPI intégré
Base de données	PostgreSQL 16	Relationnel, transactions, JSONB, contraintes
Cache / temps réel	Redis	Sessions, ZSET pour classements, rate limiting, pub/sub
Stockage fichiers	MinIO (compatible S3)	Images de stéganographie, avatars
Exécution du code	Runners isolés (gVisor ou Firecracker, ou Judge0 / isolate)	Sécurité et mesure fiable
Génération d'assets	Service Python (Pillow, `qrcode`, `stegano`)	Génération par seed des images et QR codes

Couche	Choix	Justification
CI/CD	GitHub Actions + registre de conteneurs	Voir section 11

L'architecture est un **monolithe modulaire** (modules : auth, users, challenges, scoring, teams, events, bosses, economy, chat, admin, content) avec deux composants séparés pour des raisons de sécurité et de charge : le **service Runner** (exécution du code) et le **service Assets** (génération des défis visuels).

5.3 Flux de soumission d'un défi

1. Le front envoie `POST /api/v1/challenges/{id}/submissions` (code + langage).
2. Le backend vérifie la session, les prérequis, le statut (event ouvert ? essai déjà consommé ?) et enregistre la soumission (`pending`).
3. Un job est publié dans la file ; un runner libre le consomme.
4. Le runner exécute le code dans un conteneur éphémère, sans réseau, avec limites de ressources, et renvoie résultats des tests et temps CPU.
5. Le backend calcule le score, met à jour tentatives, meilleurs scores, classements, PV, monnaie, dans **une transaction**.
6. Le résultat est poussé au client en WebSocket (avec repli sur polling).

6. Modèle de données (principales tables)

Table	Contenu clé
`users`	id, email, pseudo, password_hash, email_verified_at, status, created_at, deleted_at
`roles` , `permissions` , `role_permissions` , `user_roles`	RBAC
`sessions`	id, user_id, refresh_token_hash, user_agent, ip, expires_at, revoked_at
`password_reset_tokens`	user_id, token_hash, expires_at, used_at
`themes`	id, name, slug, region_lore_id
`challenges`	id, theme_id, type, difficulty, title, statement, base_points, perf_max, t_opt, t_max, is_event, is_boss, status

Table	Contenu clé
`challenge_versions`	challenge_id, version, tests (chiffrés), reference_solution (chiffrée)
`challenge_prerequisites`	challenge_id, required_challenge_id / required_level / required_lesson_id
`lessons`, `lesson_progress`	cours et avancement
`events`, `event_participants`	starts_at, ends_at, type (standard, battle_royale), max_attempts
`battle_rounds`, `round_results`	round_number, eliminated_id
`bosses`, `boss_phases`	max_hp, phase order, damage_rule
`player_health`	user_id (ou team_id), hp, ko_until, last_regen_at
`teams`, `team_members`	name, tag, captain_id, role, joined_at
`submissions`	id, challenge_id, user_id, team_id, language, code, status, cpu_time_ms, tests_passed, score, created_at
`best_scores`	(user_id / team_id, challenge_id) → score, submission_id
`wallets`, `transactions`	registre immuable, reason, idempotency_key
`shop_items`, `inventory`	objets et possessions
`lore_chapters`, `lore_progress`	chapitres et déblocage
`chat_channels`, `chat_messages`, `chat_reports`, `chat_sanctions`	chat et modération
`audit_logs`	actor_id, action, target, metadata, created_at

Règles d'intégrité : clés étrangères partout ; contrainte d'unicité d'un seul essai par (event, joueur ou équipe) ; index sur (`challenge_id` , `score` DESC) pour les classements ; migrations versionnées et réversibles (Prisma Migrate ou TypeORM).

7. API

API REST JSON versionnée (`/api/v1`), documentée en **OpenAPI 3** (Swagger UI), complétée par un canal WebSocket pour le temps réel. Pagination par curseur, codes HTTP standards, format d'erreur unique (`code` , `message` , `details`).

Domaine	Endpoints principaux
Auth	`POST /auth/register` , `/auth/login` , `/auth/logout` , `/auth/refresh` , `/auth/forgot-password` , `/auth/reset-password` , `/auth/verify-email`
Utilisateur	`GET/PATCH /me` , `DELETE /me` , `GET /me/export`
Défis	`GET /challenges` (filtres), `GET /challenges/{id}` , `POST /challenges/{id}/submissions` , `GET /submissions/{id}`
Cours	`GET /lessons` , `POST /lessons/{id}/complete`
Classements	`GET /leaderboards/players` , `/leaderboards/teams` , `/leaderboards/events/{id}`
Équipes	`POST /teams` , `GET /teams/{id}` , `POST /teams/{id}/invitations` , `POST /teams/{id}/members` , `DELETE /teams/{id}/members/{userId}`
Events	`GET /events` , `POST /events/{id}/join` , `POST /events/{id}/start`
Boss	`GET /bosses` , `POST /bosses/{id}/phases/{phaseId}/submissions` , `GET /me/health`
Économie	`GET /me/wallet` , `GET /shop` , `POST /shop/{itemId}/purchase`
Lore	`GET /lore/chapters`
Chat	WebSocket `chat:*` + `GET /channels/{id}/messages`

Domaine	Endpoints principaux
Admin	`/admin/users`, `/admin/roles`, `/admin/challenges`, `/admin/events`, `/admin/audit-logs`

Contraintes transverses : rate limiting par IP et par compte (plus strict sur login, reset, soumissions), idempotence des achats et des soumissions (en-tête `Idempotency-Key`), validation stricte des entrées (DTO), permissions vérifiées par garde RBAC sur chaque route.

8. Bac à sable d'exécution du code

C'est le composant le plus critique : il exécute du code arbitraire envoyé par des inconnus.

- Un conteneur **éphémère par exécution**, détruit à la fin ; image minimale par langage.
- **Aucun accès réseau**, système de fichiers en lecture seule (sauf un répertoire temporaire limité), utilisateur non root, profil seccomp restrictif, capabilities supprimées.
- Limites : CPU (cgroups), mémoire, nombre de processus, taille de sortie, durée maximale (timeout dur).
- Isolation renforcée par noyau utilisateur (gVisor) ou micro-VM (Firecracker) ; les runners tournent sur des hôtes dédiés, séparés de la base de données.
- Les tests cachés et solutions de référence ne sont jamais envoyés au navigateur ; ils sont injectés côté runner.
- Détection d'abus : taille du code, tentatives d'appels système interdits, charge anormale → sanction et journalisation.
- Tests de pénétration dédiés (fork bomb, lecture de fichiers, sortie réseau, épuisement mémoire).

9. Sécurité et gestion des données sensibles

Données sensibles : e-mails, mots de passe, jetons, tests cachés et solutions, clés d'API, secrets d'infrastructure.

- **Mots de passe** : hachés avec **Argon2id** (jamais stockés ni journalisés).
- **Transport** : HTTPS obligatoire, HSTS, TLS 1.2+ ; WebRTC chiffré (DTLS-SRTP).
- **Au repos** : chiffrement du disque de la base, chiffrement applicatif (AES-256-GCM) des tests et des solutions de référence ; clés gérées hors du dépôt.

- **Secrets** : jamais dans Git ; variables d'environnement injectées via Docker secrets, GitHub Actions secrets ou un coffre (Vault). Fichier `.env.example` sans valeur réelle. Analyse des secrets en CI (gitleaks).
- **Jetons de session et de réinitialisation** : stockés uniquement sous forme hachée en base.
- **Protection web** : CSP stricte, cookies `HttpOnly` + `Secure` + `SameSite=Lax`, protection CSRF, CORS restreint, échappement des sorties (anti-XSS), requêtes paramétrées (anti-injection SQL), en-têtes de sécurité (Helmet).
- **Anti-triche** : instances de défis générées par seed, tests cachés, comparaison de similarité de code entre joueurs (soupçons de plagiat), contrôle des soumissions multi-comptes.
- **RGPD** : consentement et information, minimisation, droit d'accès (export), droit à l'effacement (4.1), registre des traitements, conservation limitée des journaux (par ex. 12 mois).
- **Audit** : toute action d'administration et tout changement de rôle sont journalisés.
- Aucune donnée sensible dans les logs ; le masquage est automatique pour les champs `password`, `token`, `email`.

10. Gestion de la session

- **Access token** JWT court (15 min) et **refresh token** opaque (30 jours) à rotation : chaque rafraîchissement invalide l'ancien jeton ; la réutilisation d'un jeton déjà utilisé révoque toute la famille de sessions.
- Les deux jetons sont transportés par cookies `HttpOnly` ; le front n'y accède jamais en JavaScript.
- Sessions listées dans le profil (appareil, date, IP approximative), révocables individuellement.
- Inactivité : expiration glissante ; événements sensibles (changement de mot de passe, suppression de compte) exigent une **réauthentification** récente.
- Les connexions WebSocket sont authentifiées à l'ouverture et fermées à la révocation de la session.

11. Conteneurisation, déploiement et versionnage

11.1 Conteneurisation

- Un `Dockerfile` par service (front, backend, assets, runner), en **multi-stage**, images légères, utilisateur non root, `HEALTHCHECK`.

- `docker-compose.yml` pour le développement : front, back, PostgreSQL, Redis, MinIO, LiveKit, MailHog (e-mails de test), runner.
- Variables de configuration par environnement (dev, test, staging, prod) ; volumes nommés pour la base et le stockage.
- Images **multi-architecture** (amd64 et arm64) pour fonctionner sur tous les postes de développement.
- En production : orchestration Docker Compose sur serveur unique au départ, évolutive vers Kubernetes si la charge l'exige.

11.2 Versionnage

- **Git** avec dépôt unique (monorepo) : `apps/web`, `apps/api`, `services/runner`, `services/assets`, `infra/`, `docs/`.
- Stratégie de branches : `main` (protégée, toujours déployable), branches de fonctionnalité `feat/…`, correctifs `fix/…` ; fusion par **pull request** avec relecture obligatoire et CI verte.
- Commits au format **Conventional Commits** ; versions sémantiques (SemVer) ; changelog généré automatiquement ; tags de release.
- Les **défis eux-mêmes sont versionnés** (`challenge_versions`) pour que les scores restent comparables et que les changements d'énoncé soient tracés.
- Migrations de base versionnées et rejouables.

11.3 Intégration et déploiement continus

À chaque pull request : lint, vérification des types, tests unitaires, tests d'intégration, build des images, analyse de dépendances et de secrets. Sur `main` : tests de non-régression et E2E, publication des images, déploiement automatique sur staging, puis déploiement manuel validé en production.

12. Stratégie de tests

Niveau	Outils	Contenu
Unitaires	Vitest / Jest (front et back), pytest (assets)	Calcul du score, règles de prérequis, régénération de PV, ledger monétaire, règles d'élimination du battle royale, validation des mots de passe

Niveau	Outils	Contenu
Intégration	Jest + Testcontainers (PostgreSQL, Redis)	Endpoints API, transactions, RBAC, migrations
Non-régression	Jeu de **défis de référence**	Chaque défi a des solutions « golden » (correcte lente, correcte optimale, incorrecte) dont les résultats et les scores attendus sont figés ; toute modification du moteur de scoring ou du runner qui change un résultat fait échouer la CI. Tests rejoués à chaque release
Fonctionnels / E2E	Playwright	Parcours complets : inscription → confirmation → cours → défi → classement ; mot de passe oublié ; suppression de compte ; création d'équipe et édition collaborative ; event à 1 essai
Sécurité	OWASP ZAP, tests d'évasion de sandbox, `npm audit`	Injection, XSS, CSRF, contrôle d'accès, abus du runner
Charge	k6	Pic de soumissions lors d'un event, 500 connexions WebSocket simultanées
Accessibilité	axe-core	Contrastes, navigation clavier

Objectifs : couverture ⩾ 80 % sur la logique métier (scoring, économie, élimination), 100 % des parcours critiques couverts en E2E, aucune régression bloquante en production.

13. Cross-plateforme

- Application web **responsive** (mobile, tablette, desktop) compatible avec les deux dernières versions de Chrome, Firefox, Safari et Edge.
- **PWA** : installable, mode hors ligne limité (consultation des cours déjà chargés), notifications (events, fin de KO).
- Sur mobile, l'éditeur de code adopte une disposition adaptée (clavier à symboles, plein écran) ; les défis longs restent jouables mais un avertissement recommande un écran plus large.

- Le projet se développe et se lance à l'identique sous Windows, macOS et Linux grâce à Docker.
- Une application native (Capacitor) pourra être envisagée après la v1.

14. Exigences non fonctionnelles

Domaine	Exigence
Performance	Réponse API p95 < 300 ms (hors soumissions) ; résultat d'une soumission < 10 s dans 95 % des cas
Capacité	1 000 utilisateurs actifs simultanés, 100 soumissions/min en pic, extensible par ajout de runners
Disponibilité	99,5 % ; sauvegardes quotidiennes de la base, restauration testée trimestriellement
Sécurité	Conformité OWASP ASVS niveau 2 visée
Accessibilité	WCAG 2.1 AA
Internationalisation	Interface en français ; architecture prête pour l'anglais (fichiers de traduction)
Observabilité	Logs structurés, métriques (Prometheus), tableaux de bord (Grafana), alertes sur file de jobs et erreurs
Maintenabilité	Code typé, revues obligatoires, documentation d'architecture à jour

15. Feuille de route

1. **Phase 0 – Fondations** : dépôt, CI, Docker Compose, schéma de base, authentification, rôles, design system.
2. **Phase 1 – MVP** : défis de tri, bac à sable, scoring, classements joueurs, thématiques, administration des défis, tests unitaires et E2E de base.
3. **Phase 2 – Contenu et social** : didacticiels, prérequis, défis de débogage et de décryptage, équipes et édition collaborative, events à 1 essai.
4. **Phase 3 – Gamification** : boss, points de vie, monnaie et boutique, succès, lore, son, finalisation de la DA.

5. **Phase 4 – Avancé** : stéganographie et QR codes cachés, chat textuel, battle royale, chat vocal, PWA complète.
6. **Phase 5 – Consolidation** : tests de charge, audit de sécurité, accessibilité, documentation, lancement.

16. Risques et points de vigilance

Risque	Mitigation
Évasion du bac à sable	gVisor/Firecracker, hôtes dédiés, tests d'intrusion, mises à jour régulières
Mesure de temps instable (scores injustes)	Temps CPU, médiane de 5 exécutions, tolérance, calibrage par défi
Triche (partage de solutions, comptes multiples)	Instances par seed, tests cachés, détection de similarité, limites par compte
Charge lors d'un event ou d'un battle royale	File de jobs, autoscaling des runners, pré-chauffage des conteneurs, tests de charge
Inflation de la monnaie	Puits de dépense (cosmétiques, potions), plafonds de gains, suivi du registre
Modération du chat	Outils dédiés, filtres, signalements, règles de conduite
Périmètre trop large	Priorisation MoSCoW et livraison par phases
Complexité du vocal et du collaboratif	S'appuyer sur LiveKit et Yjs, ne pas développer ces briques soi-même

17. Questions ouvertes

- Quels langages de programmation supporter en v1 (Python et JavaScript proposés) ?
- Les PV s'appliquent-ils uniquement aux boss, ou aussi à certains défis standard ?
- Le battle royale est-il uniquement en équipe, uniquement en solo, ou les deux ?
- Le chat vocal est-il indispensable à la v1 ou peut-il être une intégration externe (Discord) dans un premier temps ?
- Quel public principal : étudiants, entreprises, grand public ? (impacte le ton, la modération et le RGPD, notamment pour les mineurs)
- Hébergement cible (cloud public, serveur dédié, infrastructure de l'établissement) ?

Annexe – Traçabilité des exigences

Exigence initiale	Section
Défis de tri, scoring, classement, events, boss, PV, débogage, décryptage, stéganographie, QR code	4.2 à 4.5
Interface web, cross-plateforme	5, 13
Conteneurisation	11.1
Base de données, API, backend	5, 6, 7
Versionnage	11.2, 11.3
Données sensibles	9
Inscription, connexion, mot de passe oublié, déconnexion, suppression de compte, session	4.1, 10
Utilisateur, administrateur, rôles	3, 4.12
Tests	12
Équipes	4.6
Thématiques, prérequis, didacticiels	4.7
Lore, DA, son	4.8
Battle royale	4.11
Chat textuel et vocal	4.9
Monnaie virtuelle et récompenses	4.10